

OctaC: A Language Specification

Designing a Hybrid Language for Numerical and Matrix Computation

Syed Taha

Muhammad Usman

September 5, 2026

Contents

Foreword and acknowledgments	5
Preface	7
I Language Design	9
Conclusion	11
Bibliography	13

Foreword and acknowledgments

This text details the design of OctaC, a statically typed, C-structured language for numerical and matrix computation, along with the compiler built to implement it. The chapters assembled here document the language specification and the design decisions made along the way.

We acknowledge the support and guidance of our supervisor, Miss Sadaf Alvi, whose feedback helped shape the project goals and technical approach.

The work was carried out by Syed Taha and Muhammad Usman. It was completed as part of the Compiler Construction course at our university. For questions, bug reports, or further information about the implementation, please contact Syed Taha at s.taha.29208@khi.iba.edu.pk.

Preface

OctaC is a statically typed, C-structured language for numerical and matrix computation. Dynamically typed languages like Octave let a programmer multiply two matrices of incompatible shapes without complaint until the operation actually runs. A statically typed language can catch that mistake before a single instruction executes, but only if the compiler is taught to treat shape, not just type, as a property of a value that can be checked at compile time. OctaC brings that discipline to a C-structured surface syntax, with scalar types (`i32`, `i64`, `f32`, `f64`, `bool`) alongside their matrix and vector counterparts (`i32mat`, `f32mat`, `i32vec`, `f32vec`, `boolmat`).

This is a living book. It begins with the specification of OctaC as it stands today, and is meant to grow chapter by chapter as the compiler is built over time: the formal grammar, the lexer, the parser, the semantic analysis pass, and code generation will each be added as their own chapter once they exist. Chapter 1 is the specification as it currently stands.

This book prioritizes clarity and feasibility over completeness. Where we simplify (omit a feature, restrict a rule, choose one semantic over another), we try to say so explicitly rather than let the omission speak for itself.

We hope it serves as both a reference for the finished language and an honest record of the decisions, and trade-offs, behind it.

```
function main() -> i32 {  
    printf("Hello, world!\n");  
    return 0;  
}
```


Part I

Language Design

Conclusion

Conclusion will be updated once the project is complete....

Bibliography

